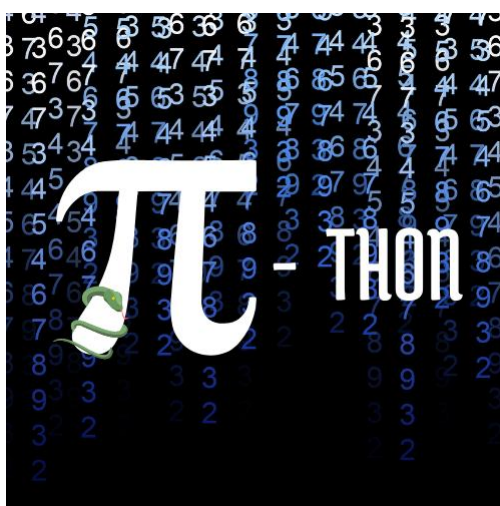




# 2025 EDITION

## PROJECT PRESENTATION



|                                              |                                       |
|----------------------------------------------|---------------------------------------|
| <b>Name of your project</b>                  | $\pi$ -THON                           |
| <b>Team Member #1 (First Name/Last Name)</b> | Arthur Jeaugey                        |
| <b>Team Member #2 (First Name/Last Name)</b> | Samuel Mopty                          |
| <b>Team Member #3 (First Name/Last Name)</b> | Paul Chevasson                        |
| <b>Level of study</b>                        | Year 12                               |
| <b>High school</b>                           | Lycée Notre-Dame, 21000 Dijon, France |
| <b>Repository Manager (NSI Professor)</b>    | M. P.Moreau                           |

# 1 / GENERAL PRESENTATION

Our project consists of developing different methods to estimate the number  $\pi$  using the Python programming language, combining mathematical approach and graphical programming with the Pygame library.

This project was born from our shared interest in mathematics and computer science, reinforced by the invitation and initiation of our teacher to participate in the Trophées NSI competition.

Our initial question was how to estimate the value of  $\pi$  precisely and efficiently using various mathematical methods, while making them visually understandable and interactive through a unique and coherent graphical display.

The main goal is to offer an intuitive interface bringing together several classic methods (Monte Carlo, Buffon, Archimedes...) allowing the user to easily visualize and compare their accuracy and performance. We thus respond to the need for a clear and interactive educational tool, useful both for teaching and for deepening the understanding of  $\pi$  and the related algorithmic concepts, and for science popularization.

In addition, the ability of our program to calculate a large number of decimals and generate text files with a desired number of decimals (example below) can prove useful for applications in scientific and cryptographic fields.

Finally, calculating a large number of decimals, especially with the Chudnovsky method, is often used as a benchmark to evaluate processor power. Our application can therefore be a good benchmarking tool (by adding certain functionalities), especially by comparing the calculation speed to the initial estimated time. This could, for example, allow comparing the performance of several machines.

```
Méthode de Gauss-Legendre
Nombre de décimales : 500 mille
Temps de calcul : 00:00:00
L'estimation de  $\pi$  est correcte !
Estimation de  $\pi$  :
3.1415926535897932384626433832795028841971693993751058209749445923078164062862089986280348253421
886593615338182796823030195203530185296899577362259941389124972177528347913151557485724245415069
407726719478268482601476990902640136394437455305068203496252451749399651431429809190659250937221
920773467221825625996615014215030680384477345492026054146659252014974428507325186660021324340881
791045334885034611365768675324944166803962657978771855608455296541266540853061434443185867697514
940924323789690706977942236250822168895738379862300159377647165122893578601588161755782973523344
82520618798531887705842972591677813149699009019211697173727847684726860849003377024242916513005
387993620162951541337142489283072201269014754668476535761647737946752004907571555278196536213239
361031029161615288138437909904231747336394804575931493140529763475748119356709110137751721008031
218654364802296780705765615144632046927906821207388377814233562823608963208068222468012248261177
510716378529023452924407736594956305100742108714261349745956151384987137570471017879573104229690
369858917544264739988228462184490087776977631279572267265556259628254276531830013407092233436577
359210031541508936793008169980536520276007277496745840028362405346037263416554259027601834840306
068691709264625484232407485503660801360466895118400936686095463250021458529309500009071510582362
```

## 2 / WORK ORGANIZATION

### Presentation of members and their roles:

- **Arthur** (Group coordinator and main developer):  
He structured the project, integrated all the methods partly coded by Samuel, created the necessary utility functions, and ensured the overall consistency of the code.
- **Samuel** (Group mathematician):  
He was responsible for researching and selecting the different mathematical methods for estimating  $\pi$ , understanding their detailed functioning, and thinking about their optimal integration into our program, while ensuring they were well explained.
- **Paul** (Graphic interface manager and videographer):  
He handled the entire graphic part of the project, in particular the visual and interactive integration of the methods into our Pygame interface, as well as the creation of the necessary graphic elements.

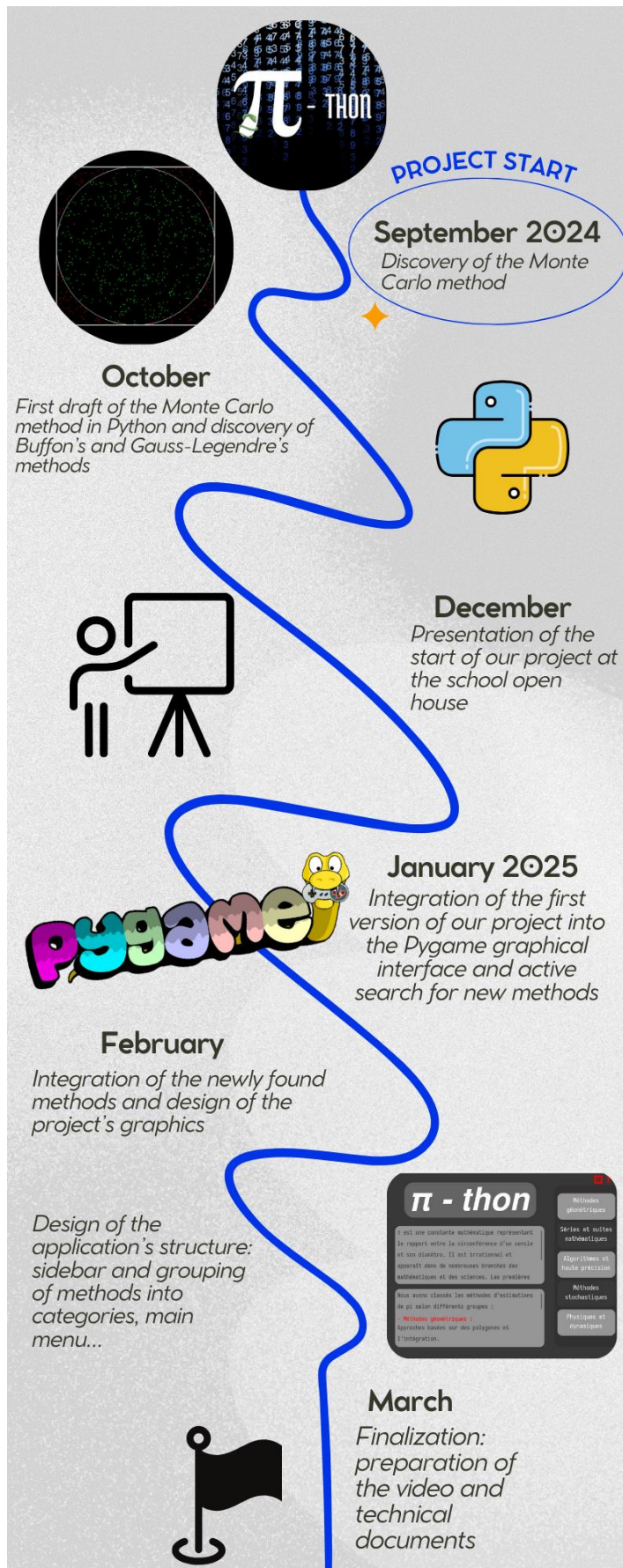
Being all three aged 16 to 17 and students at Lycée Notre-Dame, we form a group of friends passionate about computer science, used to collaborating together. The organization of the work naturally oriented itself based on each one's interests and strengths: Samuel particularly enjoys mathematics and has excellent theoretical skills in this field, Arthur is passionate about programming and is the most experienced in this area, while Paul is passionate about design and graphic aspects, fields in which he excels. This balanced distribution allowed each member to participate effectively in the project according to their skills and personal preferences, thus ensuring optimal cohesion and efficient progress.

We spent about 3 months in total on this project (In reality, we have been working on it since September, but we actively started developing the application only from December).

### Tools and software used for communication and code sharing:

-  **Discord** (daily communication and meetings to check progress)
-  **VSCode** (collaborative code editing)
-  **Cloud OneDrive** (direct and synchronized sharing of the project folder for efficient and continuous code management)

### 3 / PROJECT STEPS



We discovered the Monte Carlo method at the beginning of the year, even before having a precise project idea. This method immediately fascinated us with its simplicity and interactive aspect. With the help of our mathematics and NSI teachers, we realized that there were many other methods to estimate  $\pi$ , each based on different mathematical concepts.

After presenting our project at our school's open house, we decided to group these different approaches into a single program with graphical display under Pygame, to compare them visually and better understand they work.

Then we began a long research phase to identify the most interesting and suitable methods for our project. Some had to be discarded due to their complexity (for example, the Bailey-Borwein-Plouffe method).

Once this foundation was laid, we gradually developed and added each method, ensuring they integrated correctly into our structure. In parallel, we worked on the graphic interface with Pygame to make the display both intuitive and interactive.

Finally, we optimized the code, fixed bugs, and improved the display to ensure a smooth and functional result. After three months of work, we obtained a complete and interactive program, bringing together several  $\pi$  estimation methods in a single coherent interface.



## 4 / FUNCTIONALITY AND OPERATION

### Project status at the time of submission

The project is operational with 13 integrated  $\pi$  estimation methods displayed via a unique Pygame interface. The code structure is well defined with clear separation between functions, thus improving readability and maintainability. However, some optimizations remain to be done, such as centralizing the progress bar, but also splitting long method functions into several functions. For example, for high-performance methods, we could separate the calculation part from the graphic part into different functions. In addition, we encountered a precision limit for the most powerful methods and can currently obtain around 300 million decimals maximum with these methods. We aim to find a solution to overcome this limit and reach our set goal of calculating 1 billion decimals.

### Bug checking and ease of use

We conducted systematic tests by isolating each method to detect possible errors. Manual debugging was carried out by spotting unexpected behaviors, complemented by control logs. The `utils.py` file helped avoid code repetition and ensure better reusability. To guarantee ease of use, our application opens by running the `main.py` file. The estimated calculation time also improves the user experience by informing them of the theoretical calculation time.

### Difficulties encountered and solutions provided

- **Code redundancy:** use of the `utils.py` file to centralize some common functionalities.
- **Long calculations time on some methods:** use of the `gmpy2` library to make the calculation much faster, and envisaged progressive display of decimals to avoid too long a wait.
- **Display sometimes not optimized for fullscreen or windowed mode:** dynamic display of graphical elements using screen size for element placement rather than fixed placement.
- **Navigation:** previously, to switch methods we had to relaunch the program, so we incorporated navigation means such as a return-to-menu button.

## 5 / FUTURE DEVELOPMENTS

### Improvement ideas and new medium-term features

- **Accessibility:** add audio mode and improve color and brightness management for color-blind users.
- **code optimization:** reduce overly long files by splitting some functions, move main menu management to a more appropriate file, centralize some redundant functionalities (progress bar, time estimation...).
- **Better interactivity:** integrate progressive display of results during long calculations.
- **More decimals:** we are actively looking for a way to exceed the current 300 million decimals maximum.

### Critical analysis of the project

The project is well structured with clear module separation and well-commented code, facilitating understanding and reuse. The integration of the gmpy2 library allows high precision and faster calculations. The program is intuitive to use, with buttons enabling smooth navigation and information on calculation time for high-performance methods. Many  $\pi$  estimation methods are integrated, making our program complete. However, some functions are too long and would need to be split for more readable code. Moreover, some features such as progressive decimal display during calculation could be added.

If we were to do it again, we would better organize the project structure from the start and test some methods earlier to avoid late adjustments.

### Skills developed

This project allowed us to develop skills and knowledge in various fields: to begin with, we all gained a more precise knowledge of the history of mathematics and  $\pi$  estimation methods, but also direct knowledge such as discovering integrals and certain mathematical functions. In addition, this project improved our programming and project management skills in Python, with a better understanding of the structure to adopt. Finally, it also taught us to use a graphics library: Pygame, including learning event handling, display management, and internal functions like time management with this library.

### Promoting inclusion

For better inclusion, adding options such as audio reading and a color-blind mode would broaden the project's accessibility.